Module 6 — Microservice Architecture Report

Grad Café Analytics: Docker, RabbitMQ & Containerized Services

---

1. Docker Compose Architecture

The application is decomposed into four services defined in **docker-compose.yml**, each with a specific role, health check, and dependency ordering:

Service	Image/Build	Port	Role
db	postgres:16	5433:5432	PostgreSQL; schema auto-initialized via db/init.sql
rabbitmq	rabbitmq:3.13-management	15672:15672	Message broker; management UI at localhost:15672
web	./src/web (build)	8080:8080	Flask app; publishes tasks to RabbitMQ, serves dashboard
worker	./src/worker (build)	—	Consumes tasks from RabbitMQ; scrapes, cleans, inserts

All services share a private Docker network. PostgreSQL data is persisted in a named volume (*pgdata*). The *web* and *worker* services depend on both *db* and *rabbitmq* being healthy before starting via **condition: service_healthy**. The worker mounts *src/data/* read-only.

To start the full stack:

```
docker compose up --build
```

To seed the database from the host machine:

```
export DATABASE_URL="postgresql://postgres:<password>@localhost:5433/gradcafe"
python src/db/load_data.py
```

2. RabbitMQ Message Flow

All long-running and data-modifying work is decoupled from the web tier using RabbitMQ. Flask returns HTTP 202 immediately while the worker handles processing asynchronously.

1. User clicks **Pull Data** or **Update Analysis**.
2. Flask calls **publish_task(kind, payload)** in *publisher.py*.
3. A compact JSON message is published to durable exchange **tasks**, delivery_mode=2.
4. Flask returns **HTTP 202** immediately — {"status": "queued"}.
5. Worker's **consumer.py** receives the message from queue **tasks_q**.
6. Worker opens a DB transaction, routes by **kind**, executes the handler.
7. On success: commits and calls **basic_ack**.
8. On error: rolls back and calls **basic_nack(requeue=False)** — no infinite retries.

Entity	Name	Type	Durable
Exchange	tasks	direct	Yes
Queue	tasks_q	—	Yes
Binding	tasks → tasks_q	routing key: tasks	—

3. Worker Behavior & Task Handlers

scrape_new_data handler:

Reads last_seen watermark → scrapes GradCafe → cleans records → inserts with ON CONFLICT (url) DO NOTHING → advances watermark → commits and acks.

recompute_analytics handler:

Refreshes materialized views within the transaction → calls get_all_results() → commits and acks.

Backpressure: prefetch_count=1 ensures the worker processes one message at a time. **Watermark table** (*ingestion_watermarks*) tracks last_seen per source for idempotent incremental loads.

4. Database Initialization & SQL Safety

db/init.sql is mounted into the PostgreSQL container at */docker-entrypoint-initdb.d/00-init.sql* and executed automatically on first startup, creating the *applicants* and *ingestion_watermarks* tables. All queries continue to use parameterized psycopg2 composition — no raw string interpolation.

5. Running Application Screenshots

Web dashboard at http://localhost:8080 with real data (29,608 entries):

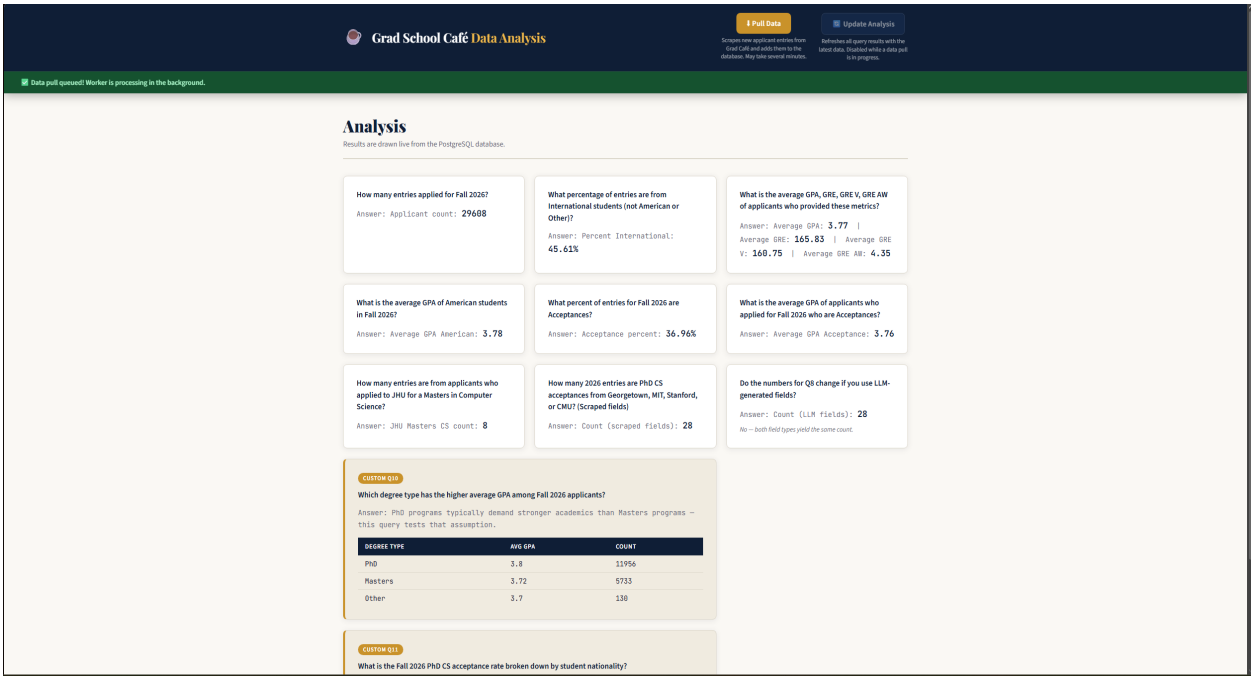

Figure 1: Flask dashboard showing live PostgreSQL data — 29,608 Fall 2026 applicants

Pull Data button — returns HTTP 202 immediately:

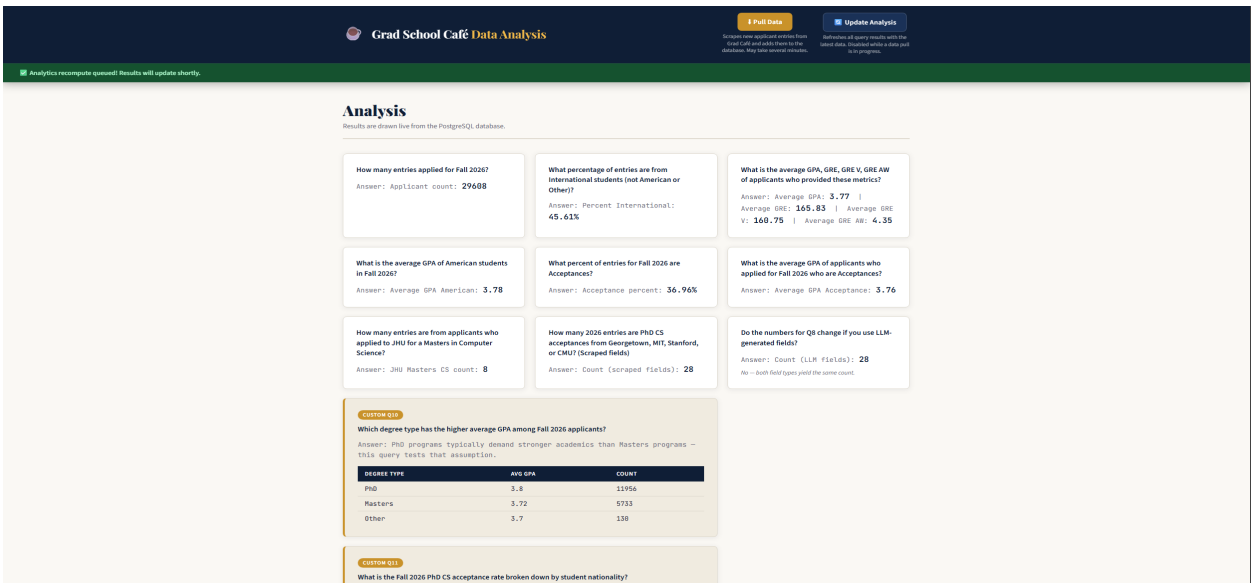

Figure 2: Green banner confirming task queued to RabbitMQ worker

Update Analysis button — returns HTTP 202 immediately:

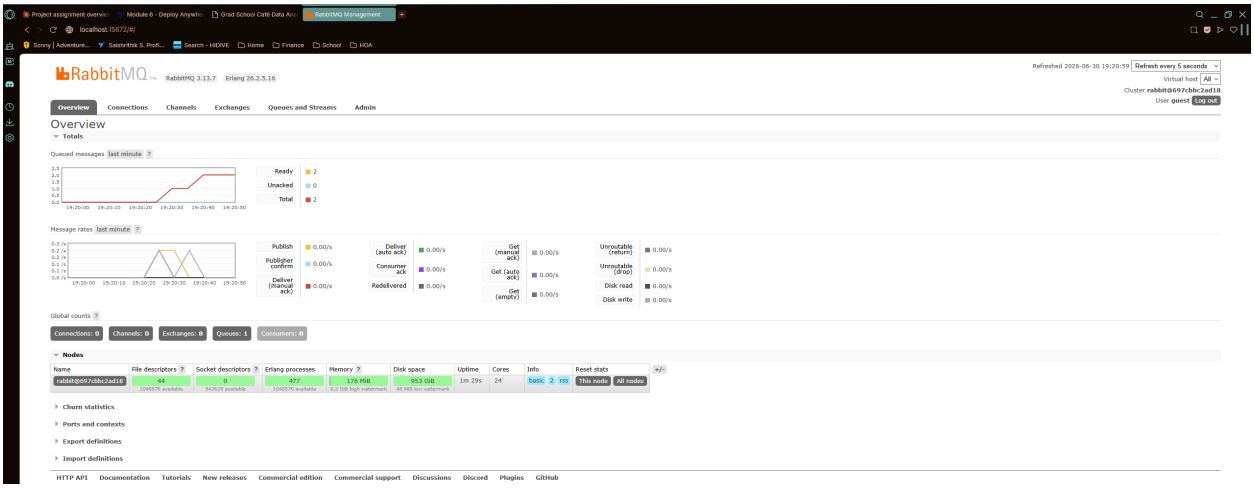

Figure 3: Analytics recompute task queued successfully

6. Docker Hub Image Publishing

Both images are publicly available at **scharfshutzer/module_6**:
Registry: https://hub.docker.com/r/scharfshutzer/module_6

Pull commands:
docker pull scharfshutzer/module_6:web
docker pull scharfshutzer/module_6:worker

7. Known Issues & Deviations

Coverage omissions (worker/etl files)

src/worker/etl/clean.py, db_config.py, and query_data.py are identical copies of web/app versions. They exist because Docker containers have isolated filesystems — COPY . . only copies files within each service directory. These are fully covered via the web/app versions. Omitted in .coveragerc with explanation in README.

src/web/app/db_config.py omitted from coverage

Tests use src/db/db_config.py directly. The web/app version is identical and omitted for the same reason.

.pylintrc similarity threshold

min-similarity-lines=10 suppresses duplicate-code for the 5-line RabbitMQ channel setup in both publisher.py and consumer.py. Moving it to a shared module is not possible without restructuring

Docker build contexts.

consumer.py main() marked pragma: no cover

main() starts a blocking RabbitMQ loop that cannot run without a live broker. All other functions including handlers, _on_message, and _open_db are fully unit tested.

DB port mapped to 5433 externally

Avoids conflict with local PostgreSQL on port 5432. Inside Docker, services use db:5432 normally. load_data.py must use localhost:5433 when seeding from the host machine.

Manual database seeding required

Data is not auto-seeded on docker compose up. Run: export DATABASE_URL=postgresql://postgres:@localhost:5433/gradcafe && python src/db/load_data.py. Schema is created automatically via db/init.sql.